EXPERIMENT 9

Shravanya Andhale
D15A-30

AIM: Implement Recurrent Neural Network (RNN) / LSTM for time series or text data

THEORY:

1. Dataset Source

Dataset: Air Quality Dataset

Source link: https://archive.ics.uci.edu/dataset/360/air+quality

The dataset contains hourly air pollution measurements collected from an Italian city.

2. Dataset Description

Total observations: approximately 9000+

Data recorded hourly

Contains pollutant and sensor readings

Target Variable Used:

CO concentration (CO_GT or equivalent column after MATLAB import)

Data Preprocessing:

Invalid values (-200) replaced with NaN

Missing values removed

Only numeric columns used

Data normalized before training

Nature of Data:

Time series data with temporal dependency

Continuous numerical values

3. Mathematical Formulation of LSTM

LSTM is a special type of Recurrent Neural Network designed to capture long-term dependencies.

Key components:

Forget Gate: decides what information to discard

Input Gate: decides what new information to store

Output Gate: determines output

Mathematical operations:

Forget Gate:
$ft = \sigma(Wf \cdot [ht\text{-}1, xt] + bf)$

Input Gate:
$it = \sigma(Wi \cdot [ht\text{-}1, xt] + bi)$

Candidate State:
$\tilde{C}t = \tanh(Wc \cdot [ht\text{-}1, xt] + bc)$

Cell State Update:
$Ct = ft * Ct\text{-}1 + it * \tilde{C}t$

Output:
$ht = ot * \tanh(Ct)$

These gates allow LSTM to retain relevant past information and discard irrelevant data.

4. Algorithm Limitations

1. Computationally expensive
2. Requires sequence formatting
3. Sensitive to hyperparameters
4. Performance affected by noisy data
5. Longer training time compared to simple models

5. Methodology / Workflow

1. Load dataset into MATLAB
2. Identify numeric columns and clean invalid values
3. Replace -200 values with NaN
4. Remove missing observations
5. Extract CO concentration as time series variable
6. Normalize data
7. Convert data into sequential format using num2cell
8. Define LSTM network architecture
9. Configure training options
10. Train model using trainNetwork
11. Generate predictions using trained model
12. Plot actual vs predicted values

6. Performance Analysis

Observed Results:

- Predicted values closely follow actual time series trends
- Model successfully captures fluctuations and patterns in CO levels
- Peaks and troughs are reasonably approximated
- Minor deviations occur due to noise and sudden spikes

Interpretation:

The LSTM model effectively learns temporal dependencies in air quality data

The predicted curve aligns well with actual values, indicating good forecasting capability

Some smoothing effect is observed in predictions, which is typical for LSTM models

From the graph:

- Strong overlap between actual and predicted curves
- Model performs well in stable regions
- Slight underestimation of extreme peaks

This demonstrates that LSTM can model complex time-dependent environmental patterns.

7. Hyperparameter Configuration

Model parameters used:

LSTM units: 100

Input size: 1

Fully connected output: 1

Training algorithm: Adam optimizer

Epochs: 100

Mini-batch size: 32

These parameters enabled the model to learn temporal dependencies effectively.

OUTPUT:

LSTM model successfully trained on time series air quality data

Forecasted values closely match actual observations

Graph visualization shows strong alignment between predicted and actual values

Model captures both short-term variations and long-term trends

matlab.mathworks.com

HOME PLOTS APPS

Search (Ctrl+Shift+Space)

New Script | New Live Script | New | Open | Go to File | Find Files | Import Data | Save Workspace | Clean Data | New Variable | Open Variable | Clear Workspace | Favorites | Clear Commands | Simulink | Layout | Set Path | Add-Ons | Settings | Parallel | Help

FILE VARIABLE CODE SIMULINK ENVIRONMENT

/ > MATLAB Drive

Files

Name

AirQualityUCI.xlsx

Workspace

Name	Value	Size	Class
ans	1×15 cell	1×15	cell
data	827×15 table	827×15	table
numericV...	1×15 logical	1×15	logical

Command Window

```
>> data = readtable('/MATLAB Drive/AirQualityUCI.xlsx');
Warning: Column headers from the file were modified to make them valid MATLAB identifiers before
creating variable names for the table. The original column headers are saved in the
VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as table variable names.
>> numericVars = varfun(@isnumeric, data, 'OutputFormat', 'uniform');
>> data{:, numericVars}(data{:, numericVars} == -200) = NaN;
>> data = rmmissing(data);
>> data.Properties.VariableNames

ans =

  1×15 cell array

  Columns 1 through 6

    {'Date'}    {'Time'}    {'CO_GT_'}    {'PT08_S1_CO_'}    {'NMHC_GT_'}    {'C6H6_GT_'}

  Columns 7 through 11

    {'PT08_S2_NMHC_'}    {'NOx_GT_'}    {'PT08_S3_NOx_'}    {'NO2_GT_'}    {'PT08_S4_NO2_'}

  Columns 12 through 15

    {'PT08_S5_O3_'}    {'T'}    {'RH'}    {'AH'}
```

Files

Name

AirQualityUCI.xlsx

Workspace

Name	Value	Size	Class
ans	1×15 cell	1×15	cell
data	827×15 table	827×15	table
layers	4×1 Layer	4×1	nnet.cnn.laye...
net	1×1 SeriesNe...	1×1	SeriesNetwork
numericV...	1×15 logical	1×15	logical
options	1×1 Training...	1×1	nnet.cnn.Trai...
series	827×1 double	827×1	double
X	826×1 cell	826×1	cell

Command Window

```
>> data.Properties.VariableNames

ans =

  1×15 cell array

  Columns 1 through 6

    {'Date'}    {'Time'}    {'CO_GT_'}    {'PT08_S1_CO_'}    {'NMHC_GT_'}    {'C6H6_GT_'}

  Columns 7 through 11

    {'PT08_S2_NMHC_'}    {'NOx_GT_'}    {'PT08_S3_NOx_'}    {'NO2_GT_'}    {'PT08_S4_NO2_'}

  Columns 12 through 15

    {'PT08_S5_O3_'}    {'T'}    {'RH'}    {'AH'}

>> series = data.CO_GT_;
>> series = normalize(series);
>> X = num2cell(series(1:end-1));
Y = num2cell(series(2:end));
>> layers = [
    sequenceInputLayer(1)
    lstmLayer(100)
    fullyConnectedLayer(1)
    regressionLayer
];
```

```
>> options = trainingOptions('adam', ...
    'MaxEpochs', 100, ...
    'MiniBatchSize', 32, ...
    'Shuffle', 'every-epoch');
>> net = trainNetwork(X, Y, layers, options);
Training on single CPU.
|=========================================================================================|
|  Epoch  |  Iteration  |  Time Elapsed  |  Mini-batch  |  Mini-batch  |  Base Learning  |
|         |             |   (hh:mm:ss)   |     RMSE     |     Loss     |      Rate       |
|=========================================================================================|
|       1 |           1 |       00:00:00 |         0.92 |          0.4 |          0.0010 |
|       2 |          50 |       00:00:00 |         0.85 |          0.4 |          0.0010 |
|       4 |         100 |       00:00:00 |         0.66 |          0.2 |          0.0010 |
|       6 |         150 |       00:00:00 |         0.71 |          0.3 |          0.0010 |
|       8 |         200 |       00:00:00 |         0.63 |          0.2 |          0.0010 |
|      10 |         250 |       00:00:00 |         0.52 |          0.1 |          0.0010 |
|      12 |         300 |       00:00:01 |         0.69 |          0.2 |          0.0010 |
|      14 |         350 |       00:00:01 |         0.56 |          0.2 |          0.0010 |
|      16 |         400 |       00:00:01 |         0.61 |          0.2 |          0.0010 |
|      18 |         450 |       00:00:01 |         0.53 |          0.1 |          0.0010 |
|      20 |         500 |       00:00:01 |         0.55 |          0.1 |          0.0010 |
|      22 |         550 |       00:00:01 |         0.74 |          0.3 |          0.0010 |
|      24 |         600 |       00:00:01 |         0.60 |          0.2 |          0.0010 |
|      26 |         650 |       00:00:02 |         0.65 |          0.2 |          0.0010 |
|      28 |         700 |       00:00:02 |         0.66 |          0.2 |          0.0010 |
|      30 |         750 |       00:00:02 |         0.63 |          0.2 |          0.0010 |
|      32 |         800 |       00:00:02 |         0.52 |          0.1 |          0.0010 |
|      34 |         850 |       00:00:02 |         0.53 |          0.1 |          0.0010 |
|      36 |         900 |       00:00:02 |         0.64 |          0.2 |          0.0010 |
|      38 |         950 |       00:00:02 |         0.78 |          0.3 |          0.0010 |
```

Command Window

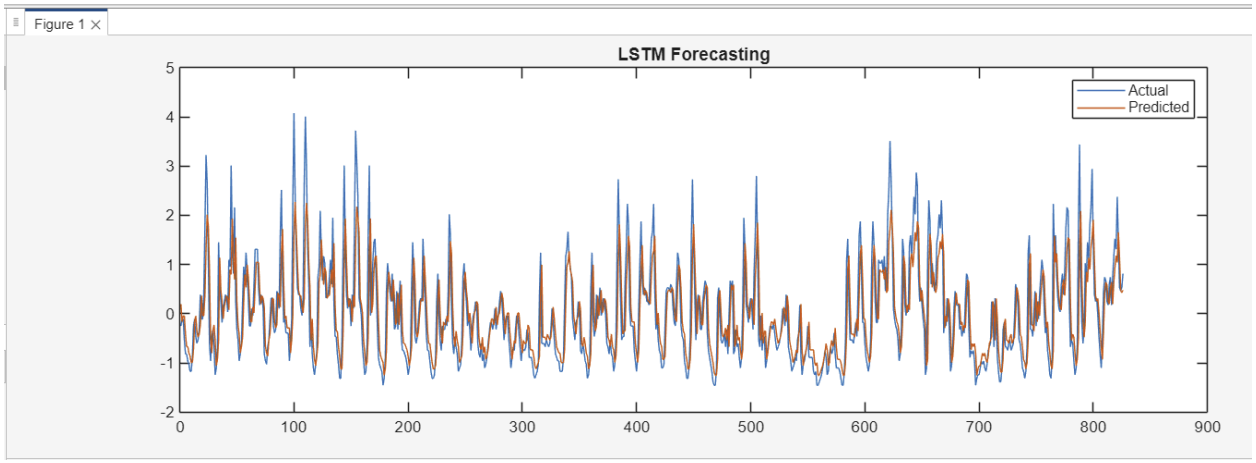
```
|        50 |         1250 |        00:00:03 |          0.69 |           0.2 |           0.0010 |
|        52 |         1300 |        00:00:03 |          0.74 |           0.3 |           0.0010 |
|        54 |         1350 |        00:00:03 |          0.63 |           0.2 |           0.0010 |
|        56 |         1400 |        00:00:03 |          0.70 |           0.2 |           0.0010 |
|        58 |         1450 |        00:00:04 |          0.63 |           0.2 |           0.0010 |
|        60 |         1500 |        00:00:04 |          0.67 |           0.2 |           0.0010 |
|        62 |         1550 |        00:00:04 |          0.55 |           0.2 |           0.0010 |
|        64 |         1600 |        00:00:04 |          0.67 |           0.2 |           0.0010 |
|        66 |         1650 |        00:00:04 |          0.49 |           0.1 |           0.0010 |
|        68 |         1700 |        00:00:04 |          0.61 |           0.2 |           0.0010 |
|        70 |         1750 |        00:00:04 |          0.45 |           0.1 |           0.0010 |
|        72 |         1800 |        00:00:05 |          0.64 |           0.2 |           0.0010 |
|        74 |         1850 |        00:00:05 |          0.46 |           0.1 |           0.0010 |
|        76 |         1900 |        00:00:05 |          0.64 |           0.2 |           0.0010 |
|        78 |         1950 |        00:00:05 |          0.86 |           0.4 |           0.0010 |
|        80 |         2000 |        00:00:05 |          0.76 |           0.3 |           0.0010 |
|        82 |         2050 |        00:00:05 |          0.56 |           0.2 |           0.0010 |
|        84 |         2100 |        00:00:05 |          0.46 |           0.1 |           0.0010 |
|        86 |         2150 |        00:00:05 |          0.55 |           0.2 |           0.0010 |
|        88 |         2200 |        00:00:06 |          0.85 |           0.4 |           0.0010 |
|        90 |         2250 |        00:00:06 |          0.42 |       8.8e-02 |           0.0010 |
|        92 |         2300 |        00:00:06 |          0.81 |           0.3 |           0.0010 |
|        94 |         2350 |        00:00:06 |          0.77 |           0.3 |           0.0010 |
|        96 |         2400 |        00:00:06 |          0.39 |       7.7e-02 |           0.0010 |
|        98 |         2450 |        00:00:06 |          0.70 |           0.2 |           0.0010 |
|       100 |         2500 |        00:00:06 |          0.72 |           0.3 |           0.0010 |
|===========================================================================================|
Training finished: Max epochs completed.
>>
```

Workspace

Name	Value	Size	Class
ans	1×15 cell	1×15	cell
data	827×15 table	827×15	table
layers	4×1 Layer	4×1	nnet.cnn.laye...
net	1×1 SeriesNe...	1×1	SeriesNetwork
numericV...	1×15 logical	1×15	logical
options	1×1 Training...	1×1	nnet.cnn.Trai...
series	827×1 double	827×1	double
X	826×1 cell	826×1	cell
Y	826×1 cell	826×1	cell
YPred	826×1 cell	826×1	cell


```
|        90 |         2250 |        00:00:06 |          0.42 |       8.8e-02 |           0.0010 |
|        92 |         2300 |        00:00:06 |          0.81 |           0.3 |           0.0010 |
|        94 |         2350 |        00:00:06 |          0.77 |           0.3 |           0.0010 |
|        96 |         2400 |        00:00:06 |          0.39 |       7.7e-02 |           0.0010 |
|        98 |         2450 |        00:00:06 |          0.70 |           0.2 |           0.0010 |
|       100 |         2500 |        00:00:06 |          0.72 |           0.3 |           0.0010 |
|===========================================================================================|
Training finished: Max epochs completed.
```


```
>> YPred = predict(net, X);
>> plot(cell2mat(Y))
hold on
plot(cell2mat(YPred))
legend('Actual','Predicted')
title('LSTM Forecasting')
>>
```



CONCLUSION:

LSTM is highly effective for time series forecasting tasks.The model successfully captured temporal patterns in air quality data.Predictions closely follow actual values, demonstrating strong learning capability.LSTM networks are suitable for environmental monitoring and forecasting applications.Deep learning methods outperform traditional models in capturing sequential dependencies.
This experiment demonstrates the practical application of LSTM networks for real-world time series forecasting problems.